Task 1.2: Powell's Method

Objective

Implement and test *Powell's method* (the derivative-free conjugate-direction method) in Python. The goal is to learn how direction-set methods build and update search directions, how the additional direction (the displacement after a cycle) accelerates progress, and how Powell compares to coordinate-wise methods (CCD) on nonconvex test problems.

Algorithm to implement

Translate the pseudocode below (the version discussed in class) into a Python implementation that uses a one-dimensional line search routine.

Algorithm 1 Powell's Method

```
function POWELL(f, x, ϵ)
    n = length(x)
    U = [basis(i, n) for i in 1 : n]
    Δ = ∞
    while Δ > ϵ do
        x′ = x
        for i in 1 : n do
            x = line_search(f, x, U[i])
        end for
        for i in 1 : n − 1 do
            U[i] = U[i + 1]
        end for
        U[n] = x − x′
        x = line_search(f, x, U[n])
        Δ = ‖x − x′‖
    end while
    return x
end function
```

You may implement the line search with golden-section search or an Armijo/backtracking scheme; document your choice.

Test functions

Apply your Powell implementation to the following 2D benchmark functions.

- **Ackley function** (2D):

$$f(x,y) = -20\exp\Big(-0.2\sqrt{\tfrac{1}{2}(x^2+y^2)}\Big) - \exp\Big(\tfrac{1}{2}(\cos(2\pi x)+\cos(2\pi y))\Big) + e + 20.$$

  (Use the standard parameterization above. Global minimum at $(0,0)$ with $f(0,0) = 0$.)

- **Branin function** (2D):

$$f(x,y) = \Big(y - \tfrac{5.1}{4\pi^2}x^2 + \tfrac{5}{\pi}x - 6\Big)^2 + 10\Big(1 - \tfrac{1}{8\pi}\Big)\cos(x) + 10.$$

  (Multiple global minima; one example (π, 2.275) with $f \approx 0.397887$.)

Starting points

Run deterministic single runs (no randomness) with these starts:

- Ackley: $x^{(0)} = (-3.0,\ -3.0)$
- Branin: $x^{(0)} = (2.0,\ 2.0)$

Experiment protocol and outputs

For each function perform one run of Powell's method (with a fixed tolerance, e.g. $\epsilon = 10^{-6}$, and reasonable max iterations). Record and save the following artifacts:

1. **Results:** in the console, display a table with columns: `function, start, method,` x_found`, f(`x_found`), iterations, time of working`. Use method='Powell'.
2. **Trajectory plot:** save '$plot\_ < nameoftestfunction > \_trajectory.png$' — contour plot of the objective with the optimization path overlaid (mark start, final point, and at least one known global minimum).
3. **Convergence plot:** save '$plot\_ < nameoftestfunction > \_convergence.png$' — plot of objective value vs iteration (or vs function evaluations) showing how $f(x^{(k)})$ decreases.
4. **Implementation:** Python file implementing Powell and the line search.

Tasks

1. Implement Powell's method in Python following the pseudocode above.
2. Implement a 1D line search routine (golden-section or backtracking/Armijo). Use it inside Powell.
3. Run Powell on Ackley and Branin with the specified starts.
4. Produce the requested files: '$plot\_ < nameoftestfunction > \_trajectory.png$', '$plot\_ < nameoftestfunction > \_convergence.png$'.
5. Using the result of a comparative checking (see the next task) write short summary (3–5 sentences) describing:
   - Which method reached a lower objective and why.
   - Which method used fewer function evaluations and work faster.
   - Qualitative differences in trajectories and sensitivity to initial points.

Comparative check

Compare Powell's results with the results of Cyclic Coordinate Descent with Acceleration Step (from Task 1.1). For this comparison:

- Use the same line-search implementation in both methods.
- Use the same test functions (Rosenbrock and Ackley) and the same starting points ($x^{(0)} = (-1.5, 2.0)$ for Rosenbrock and $x^{(0)} = (4.0,\ 1.0)$ for Ackley correspondingly)

Deliverables

- Python code implementing Powell's method and the line-search routine.
- A summary table (in the console) with results for all runs.
- Plots showing trajectories and convergence on the test function.
- A short written discussion (3–5 sentences) comparing Powell and CCD for these tests.

Additional exercises (you must complete one of them of your choice)

1. **Line-search study:** Repeat experiments using two different line-search strategies (golden-section vs backtracking) and compare performance.
2. **Direction evolution:** Save and visualize the set of basis directions U after several outer iterations to show how Powell builds a useful search basis.
3. **Noisy objective:** Add small Gaussian noise to function evaluations and test algorithm robustness (repeat runs, compute statistics).
4. **Hybrid method:** Implement a hybrid: run a few Powell cycles and then switch to CCD (or vice versa). Report whether hybridization helps on Ackley / Rosenbrock.

Notes and practical hints

- Use consistent stopping criteria across methods (same tolerance and max iterations) for a fair comparison.
- Track *function evaluations* as a primary cost measure (line-search calls can be expensive).
- For plotting trajectories on multimodal surfaces (Ackley and Branin), choose plotting domains that show both the starting point and the region around known minima.